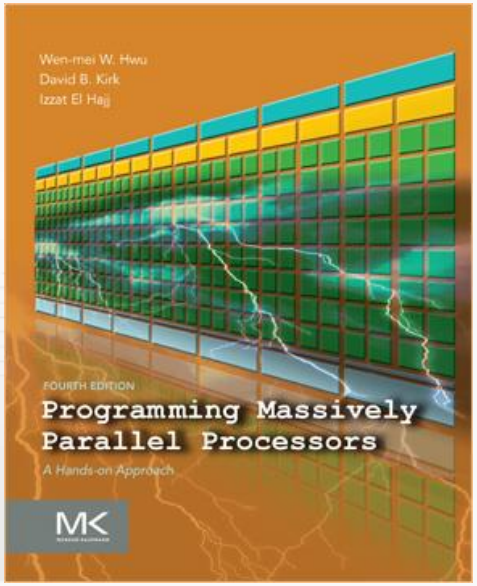
MORGAN KAUFMANN

Programming Massively Parallel Processors

A Hands-on Approach

CHAPTER 5 › Memory Architecture and Data Locality



Copyright © 2022 Elsevier

Performance Metrics

- Processor designers inform users about a processor's performance through various metrics, most notably:
  - **FLOPS Rate**: floating point operations per second
    - How much computation a processor's cores do per unit time
  - **Memory Bandwidth**: bytes per second
    - How much data the memory supplies to the cores per unit time

GPU Name	K40	M40	P100	V100
Microarchitecture	Kepler	Maxwell	Pascal	Volta
Peak FLOPS Rate (GFLOPS)	5,040	6,844	9,340	14,028
Peak Memory Bandwidth (GB/s)	288	288	732	900

- These metrics are *not a guarantee*
  - They are a *limit* of the maximum possible performance

Copyright © 2022 Elsevier

Performance Bounds

- An application can be:
  - **Compute-bound**: performance limited by the FLOPS rate
    - The processor's cores are fully utilized (always have work to do)
  - **Memory-bound**: performance limited by the memory bandwidth
    - The processor's cores are frequently idle because memory cannot supply data fast enough

GPU Name	K40	M40	P100	V100
Microarchitecture	Kepler	Maxwell	Pascal	Volta
Peak FLOPS Rate (GFLOPS)	5,040	6,844	9,340	14,028
Peak Memory Bandwidth (GB/s)	288	288	732	900
Desired FP to Memory Access Ratio (OP/B)	17.5	23.8	12.8	15.6

Number of times a byte has to be reused to amortize the memory access

Copyright © 2022 Elsevier

Example: Vector Addition

```
z[i] = x[i] + y[i];
```

- Vector addition performs 0.125 OP/B
  - 1 FP add operation for every 2 FP values (8B) loaded
  - Ignoring stores

- Vector addition is highly memory-bound
  - Did not see significant speedup because the massive number of ALUs were mostly idle
  - Nothing we can do (no opportunities for reuse)

Copyright © 2022 Elsevier

Example: Matrix-Matrix Multiplication

```
for(unsigned int i = 0; i < N; ++i) {
    sum += A[row*N + i]*B[i*N + col];
}
```

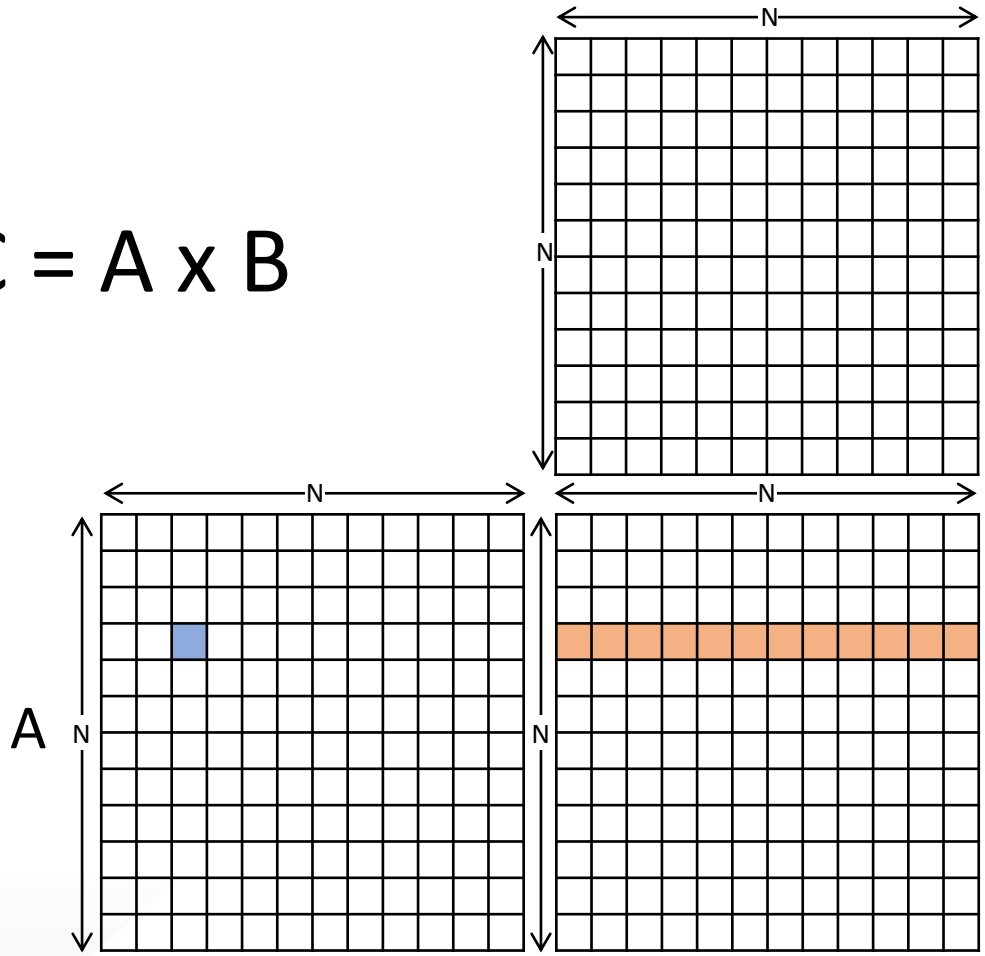
- Matrix multiplication performs 0.25 OP/B
  - 1 FP add and 1 FP mul for every 2 FP values (8B) loaded
  - Ignoring stores

- But matrix multiplication has high potential for reuse
  - For N×N square matrices
    - Data loaded: (2 input matrices)×(N^2 values)×(4 B) = $8N^2$ B
    - Operations: (N^2 dot products)(N adds + N muls each) = $2N^3$ OP
    - Potential floating point to global memory access ratio: 0.25N OP/B

Copyright © 2022 Elsevier

Reuse in Matrix-Matrix Multiplication

$C = A \times B$

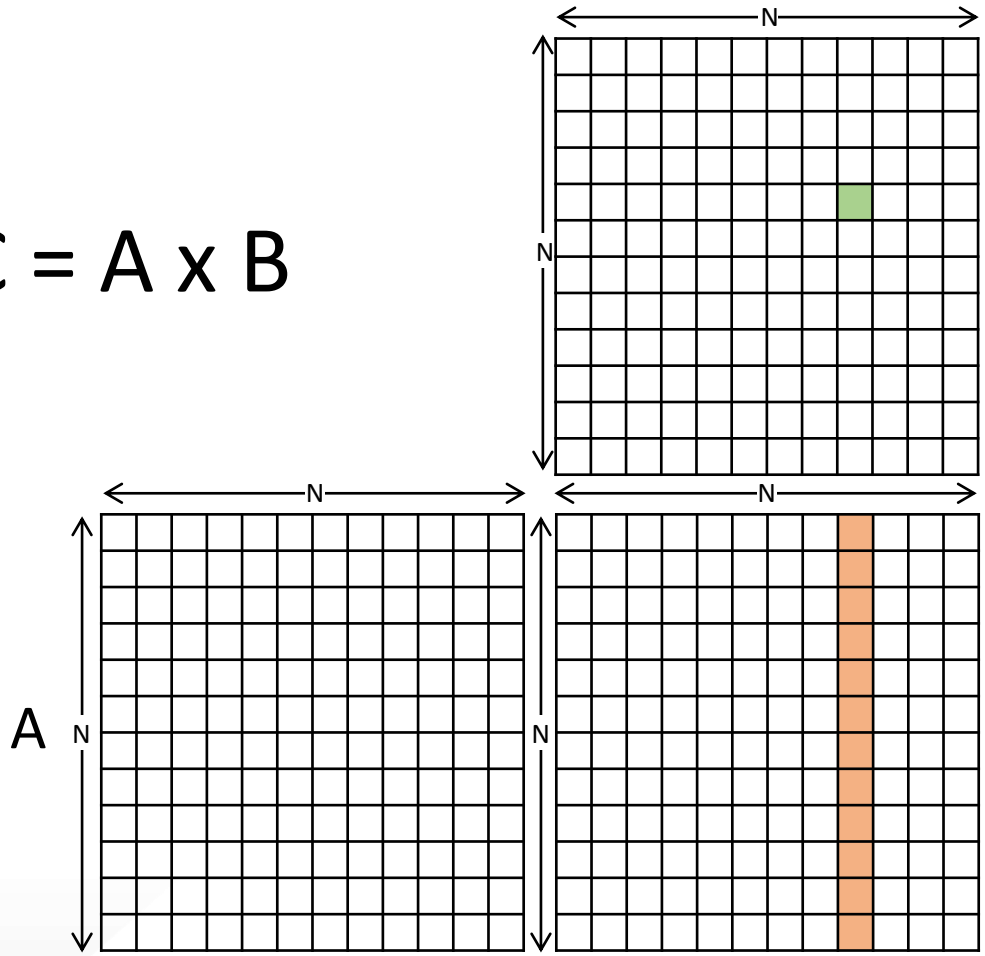


Every element of the input is used for computing and entire row/column of the output

Copyright © 2022 Elsevier

Reuse in Matrix-Matrix Multiplication

C = A x B

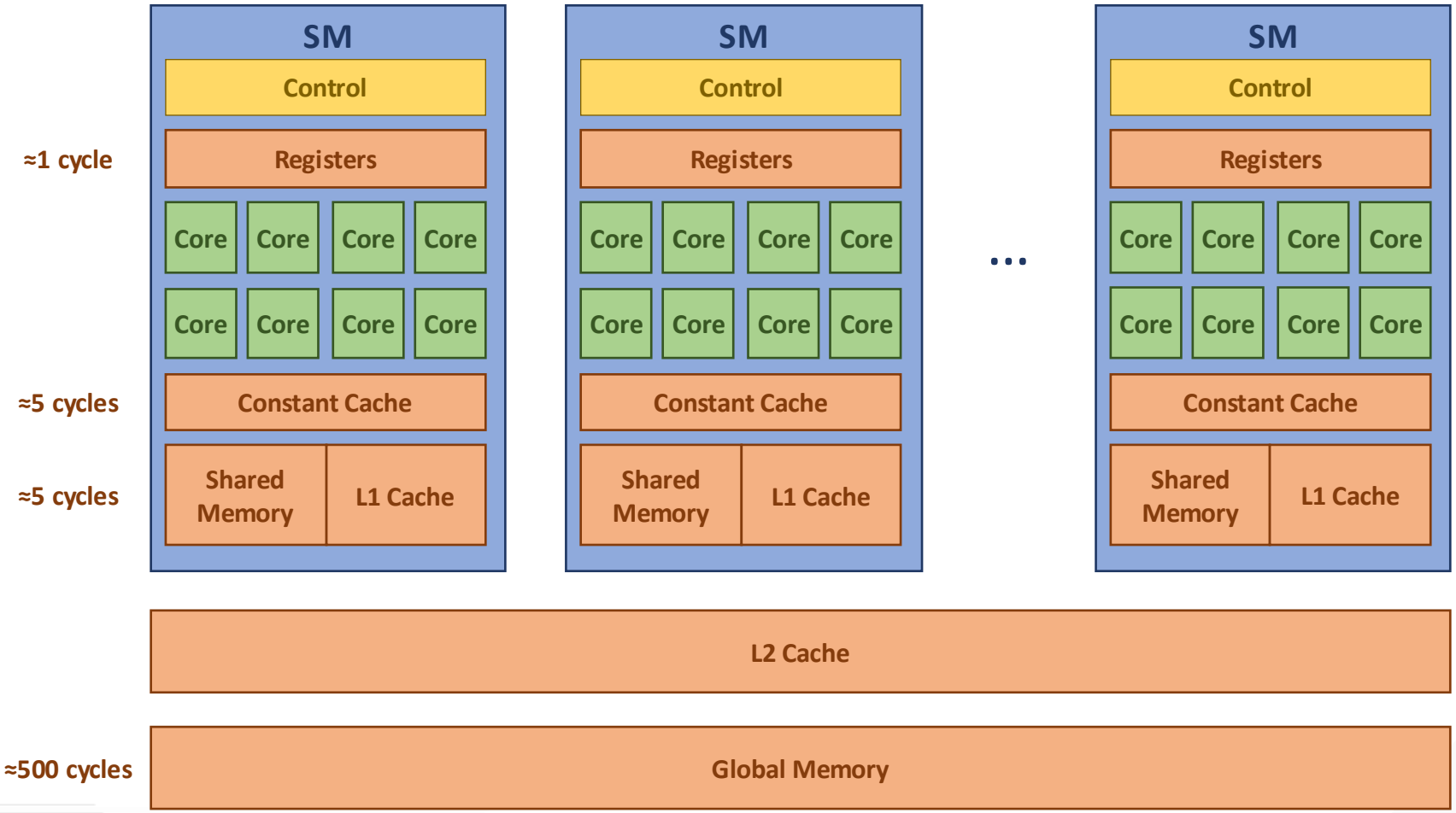


Every element of the input is used for computing and entire row/column of the output

Opportunity: Optimize memory accesses in the kernel code to reuse input data loaded from global memory as much as possible

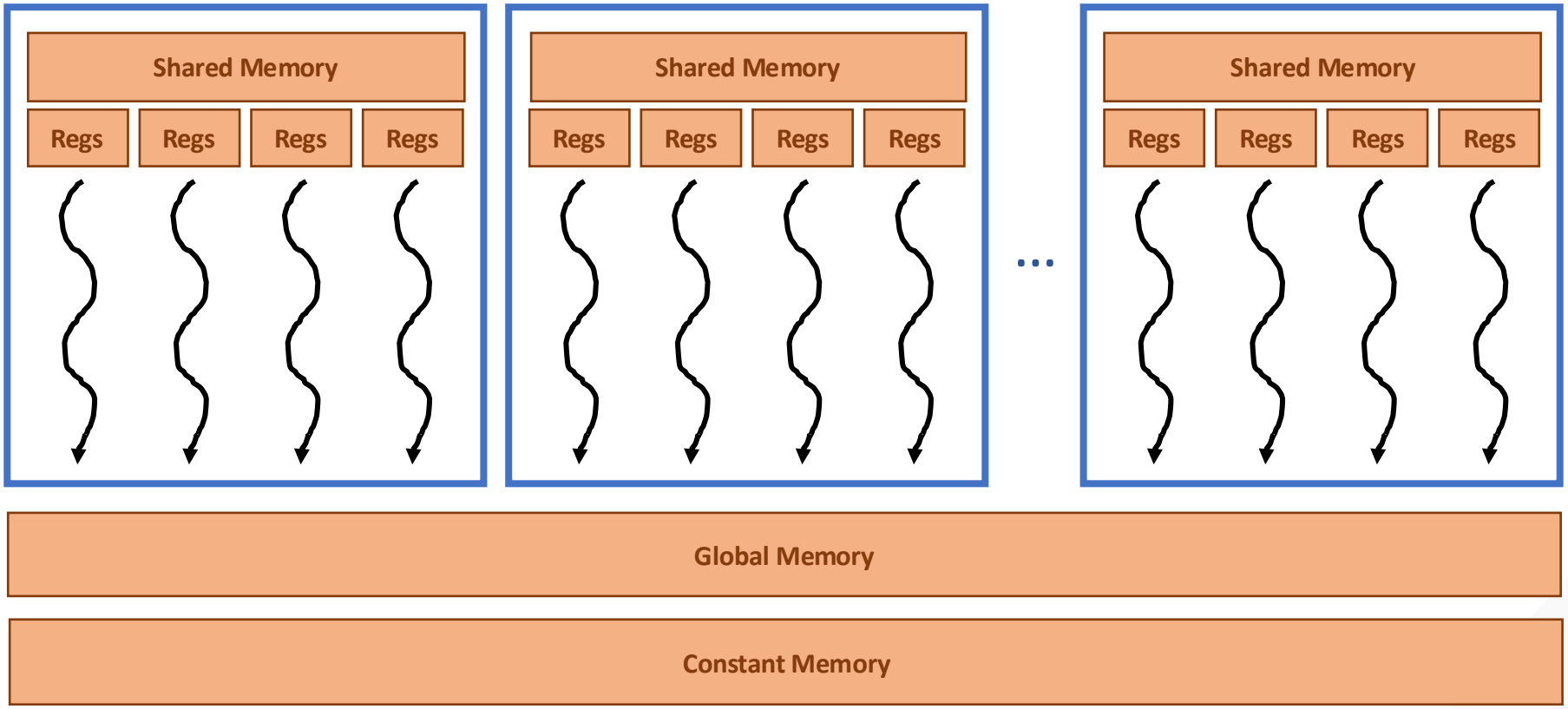
Copyright © 2022 Elsevier

Memory in the GPU Architecture



Copyright © 2022 Elsevier

Memory in the CUDA Programming Model



Copyright © 2022 Elsevier

CUDA Type Qualifies

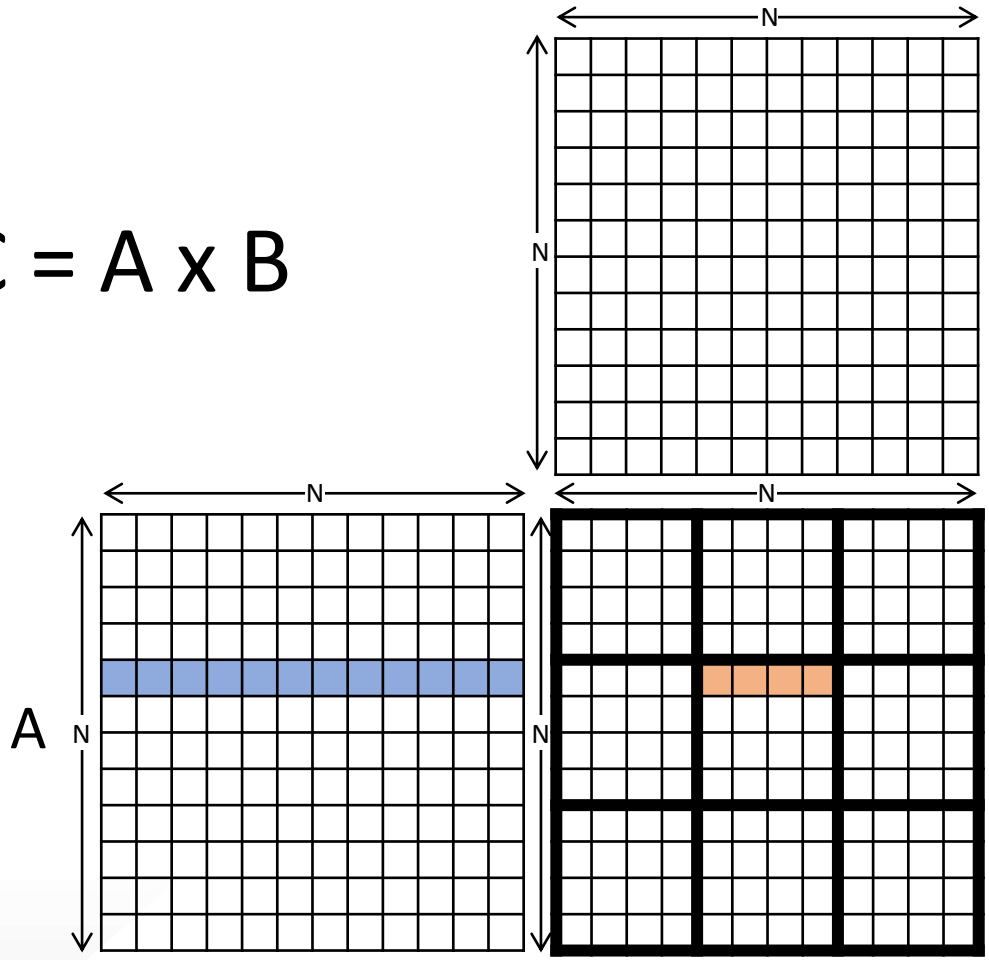
Variable declaration		Memory	Scope	Lifetime
`__device__`	`int globalVar;`	global	grid	application
`__device__ __constant__`	`int constantVar;`	constant	grid	application
`__device__ __shared__`	`int sharedVar;`	shared	block	block
	`int localVar;`	register	thread	thread
	`int localArr[N];`	global	thread	thread

- We have seen cudaMalloc(…) to also allocate data in global memory from the host
- Constant memory allocation and initialization will be discussed later
- Focus of today: **shared memory**
  - Also known as **scratchpad memory**

Copyright © 2022 Elsevier

Reuse in Matrix-Matrix Multiplication

$C = A \times B$

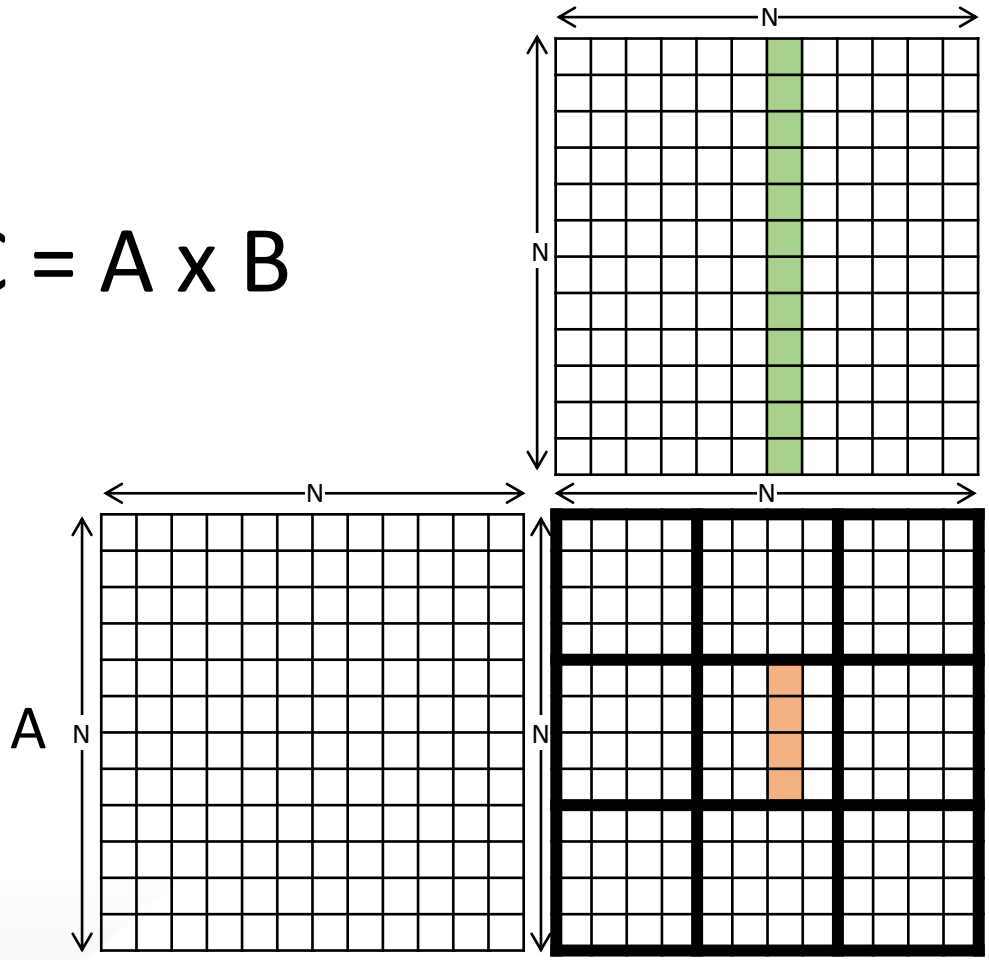


Some of the threads in the same thread block use the same input data

Copyright © 2022 Elsevier

Reuse in Matrix-Matrix Multiplication

$C = A \times B$



Some of the threads in the same thread block use the same input data

Copyright © 2022 Elsevier

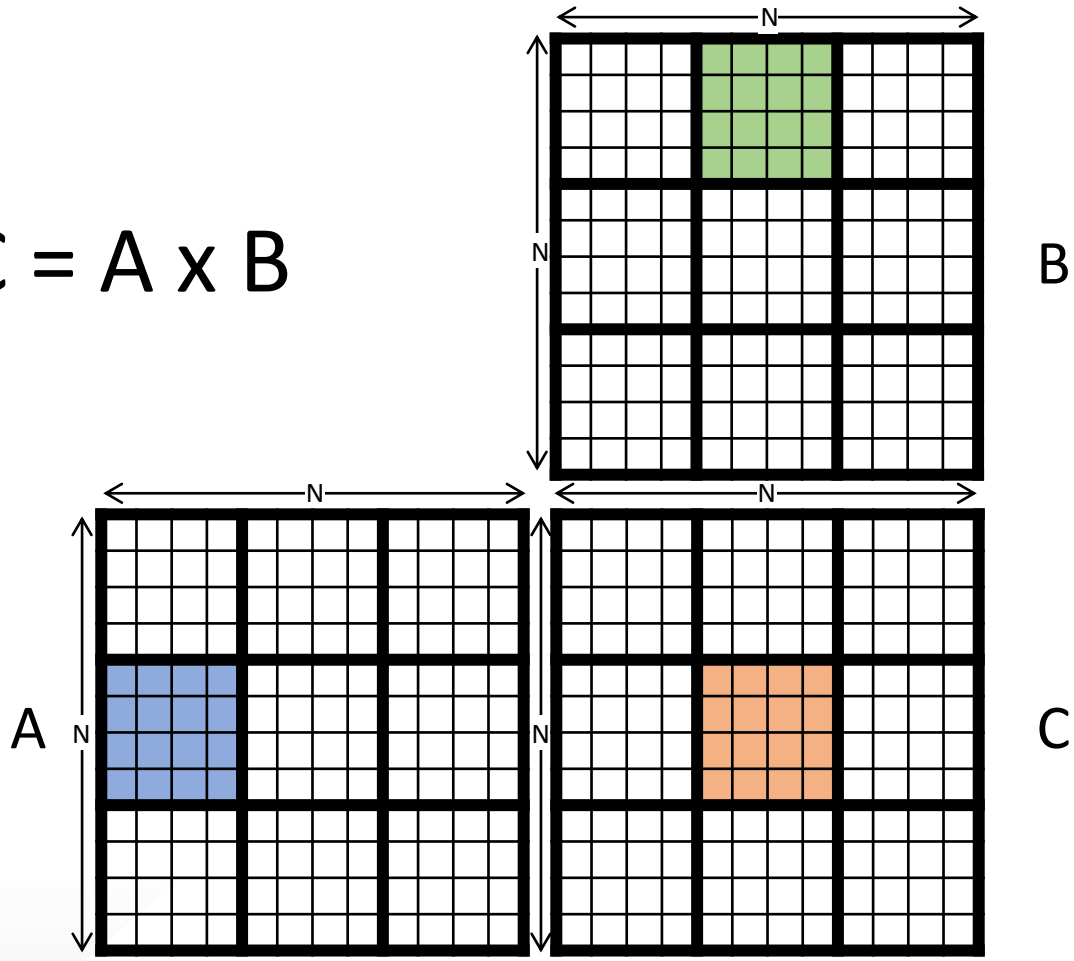
Reuse in Matrix-Matrix Multiplication

- Sometimes, we are lucky:
  - The thread finds the data in the L1 cache because it was recently loaded by another thread
- Sometimes, we are not lucky:
  - The data gets evicted from the L1 cache before another thread tries to load it
- Solution:
  - Let the threads work together to load part of the data and ensure that all threads that need it use it before loading more data
  - Use shared memory to ensure data stays close
  - Optimizing called **tiling** because divides input to tiles

Copyright © 2022 Elsevier

Tiled Matrix-Matrix Multiplication

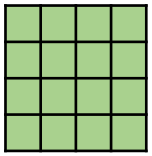
C = A x B



Step 1: Load the first tile of each input matrix to shared memory (each thread loads one element)

Copyright © 2022 Elsevier

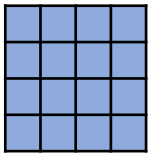
Tiled Matrix-Matrix Multiplication

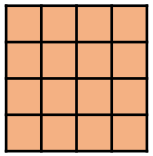


B_{tile}

$$C_{tile} = A_{tile} \times B_{tile}$$

Step 2: Each thread computes its partial sum from the tiles in shared memory (threads wait for each other to finish)

A_{tile} 

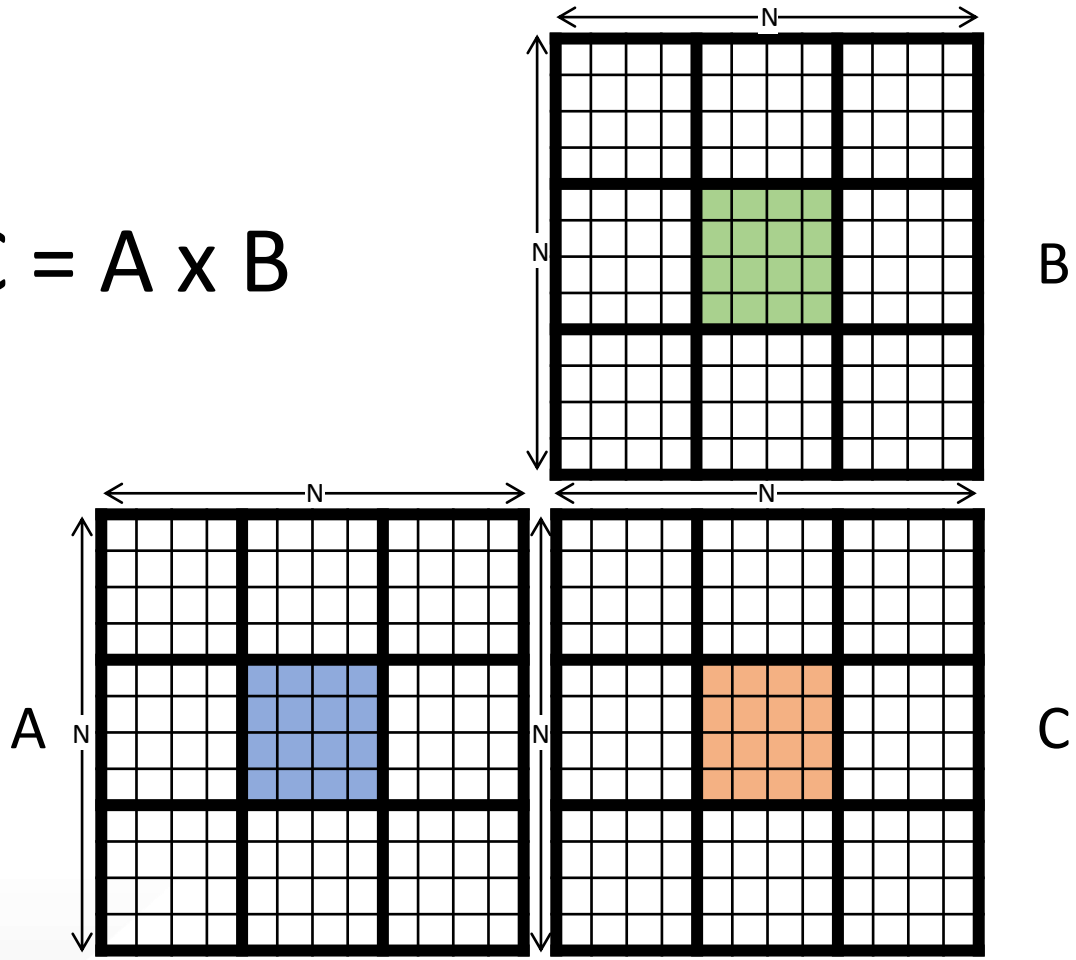


C_{tile}

Copyright © 2022 Elsevier

Tiled Matrix-Matrix Multiplication

C = A x B

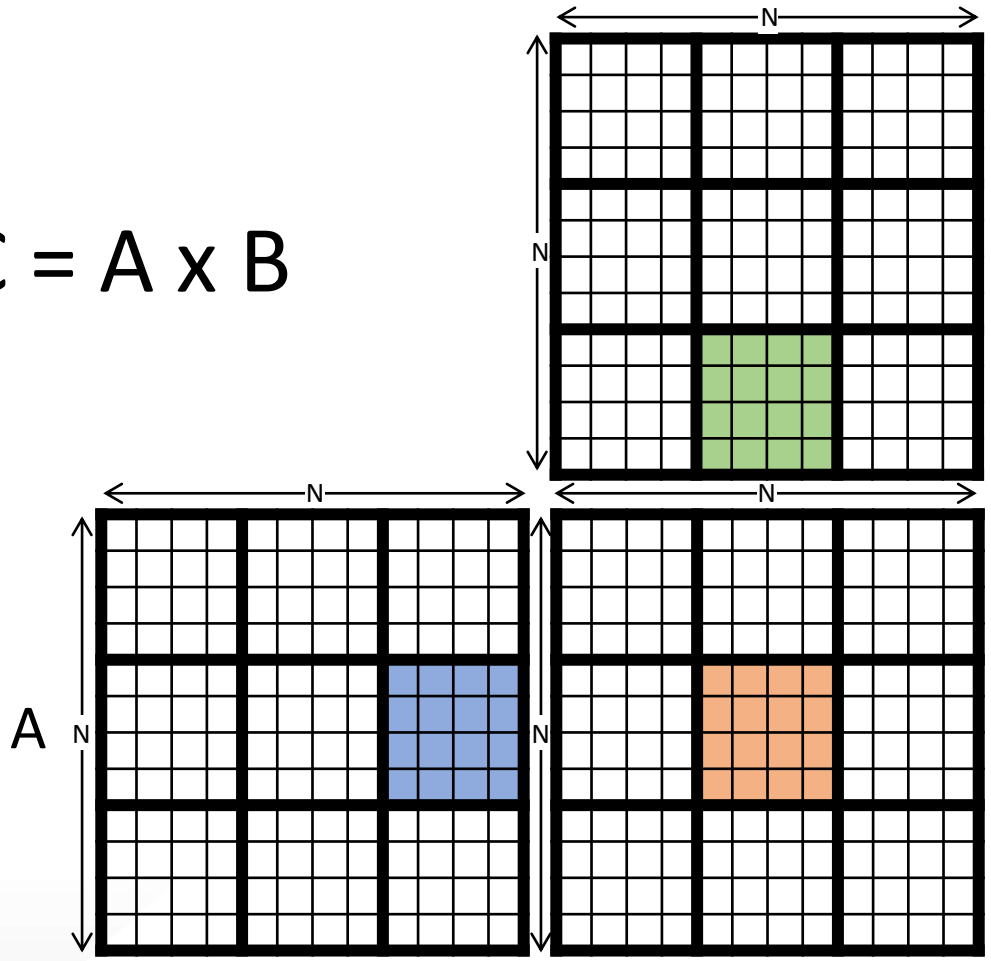


…repeat for the next tile

Copyright © 2022 Elsevier

Tiled Matrix-Matrix Multiplication

C = A x B



…and the next tile

Copyright © 2022 Elsevier

Tiled Matrix-Matrix Multiplication

```
__shared__ float A_s[TILE_DIM][TILE_DIM];
__shared__ float B_s[TILE_DIM][TILE_DIM];

unsigned int row = blockIdx.y*blockDim.y + threadIdx.y;
unsigned int col = blockIdx.x*blockDim.x + threadIdx.x;

float sum = 0.0f;

for(unsigned int tile = 0; tile < N/TILE_DIM; ++tile) {

    // Load tile to shared memory
    A_s[threadIdx.y][threadIdx.x] = A[row*N + tile*TILE_DIM + threadIdx.x];
    B_s[threadIdx.y][threadIdx.x] = B[(tile*TILE_DIM + threadIdx.y)*N + col];
    __syncthreads();

    // Compute with tile
    for(unsigned int i = 0; i < TILE_DIM; ++i) {
        sum += A_s[threadIdx.y][i]*B_s[i][threadIdx.x];
    }
    __syncthreads();

}

C[row*N + col] = sum;
```

Declare arrays in shared memory

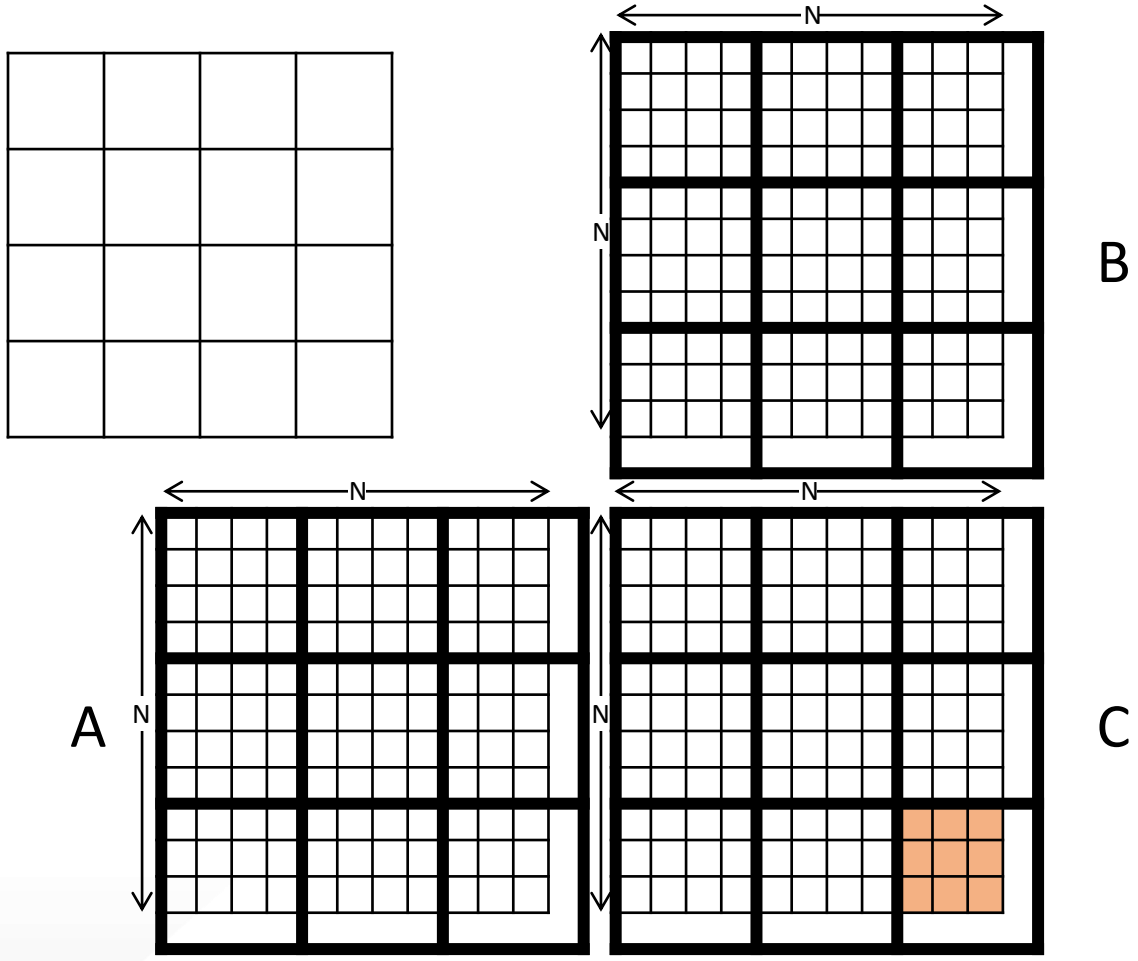
Threads wait for each other to finish loading before computing

Threads wait for each other to finish computing before loading

Copyright © 2022 Elsevier

Boundary Conditions

thread block processing colored output tile

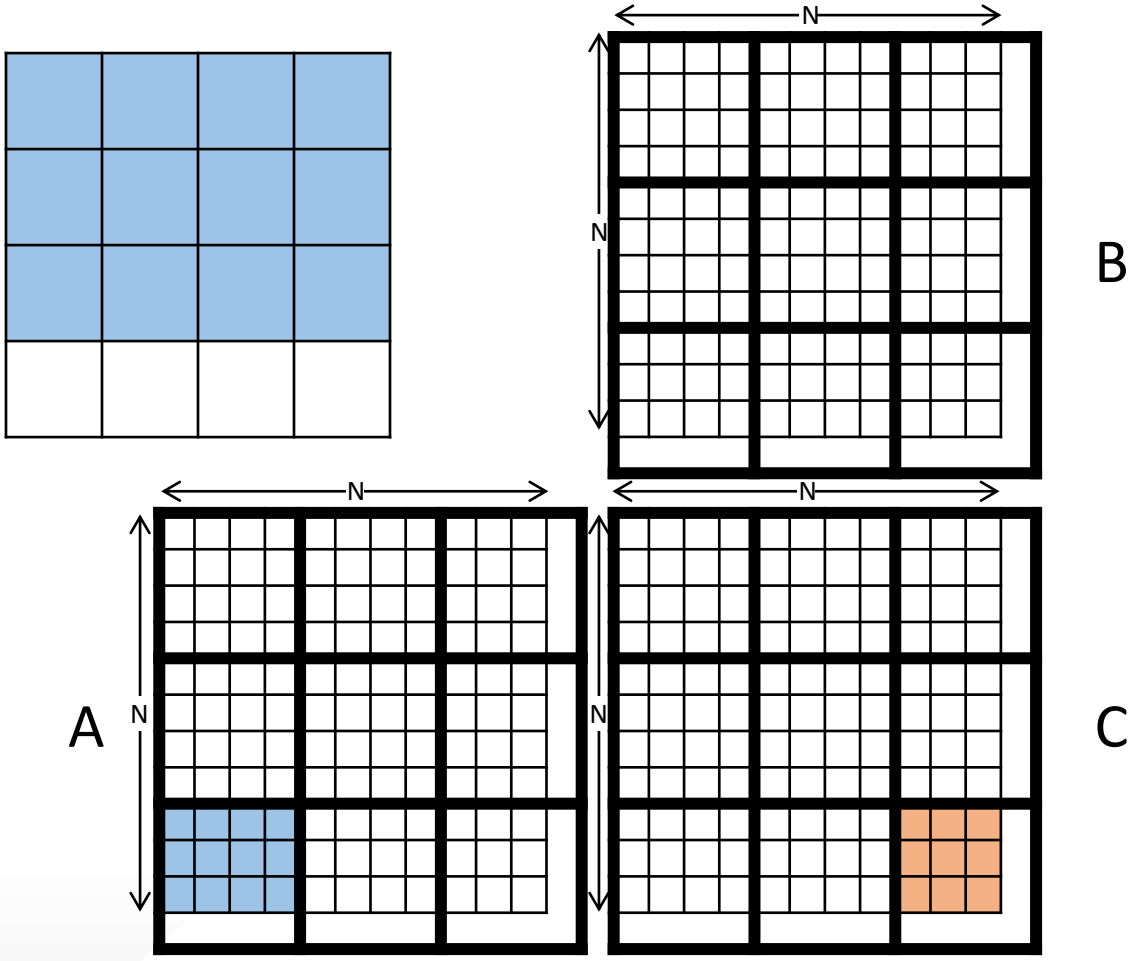


Copyright © 2022 Elsevier

Boundary Conditions

thread block processing colored output tile

threads active while loading A tile

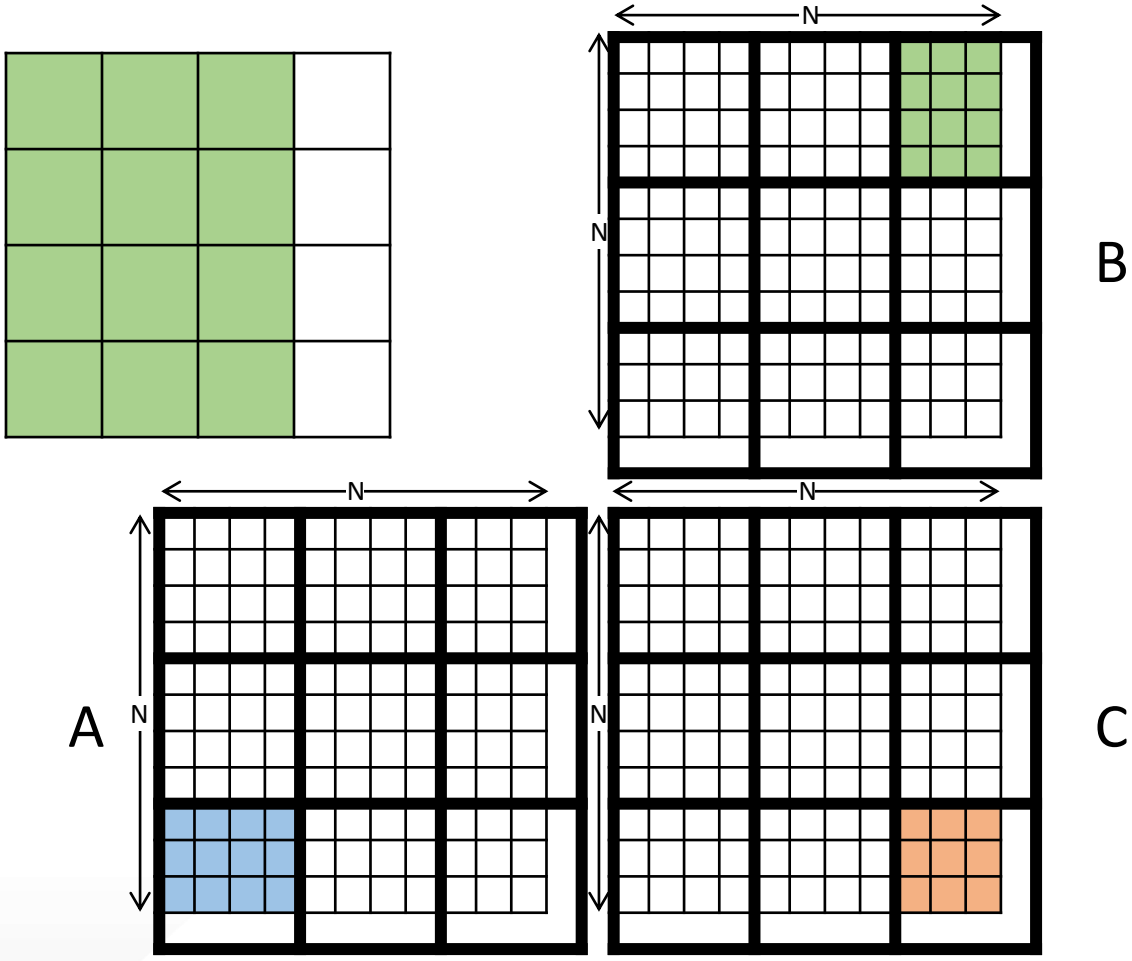


Copyright © 2022 Elsevier

Boundary Conditions

thread block processing colored output tile

threads active while loading B tile

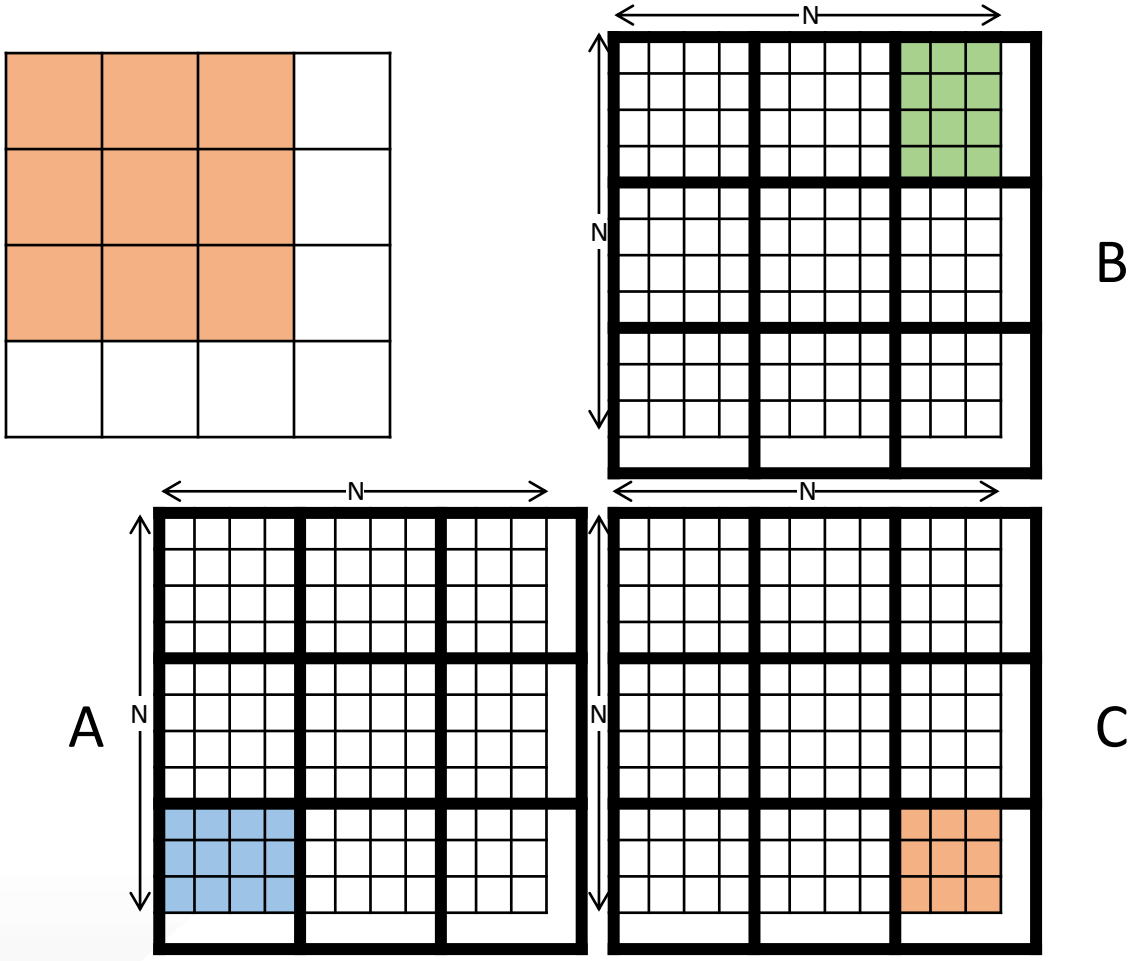


Copyright © 2022 Elsevier

Boundary Conditions

thread block processing colored output tile

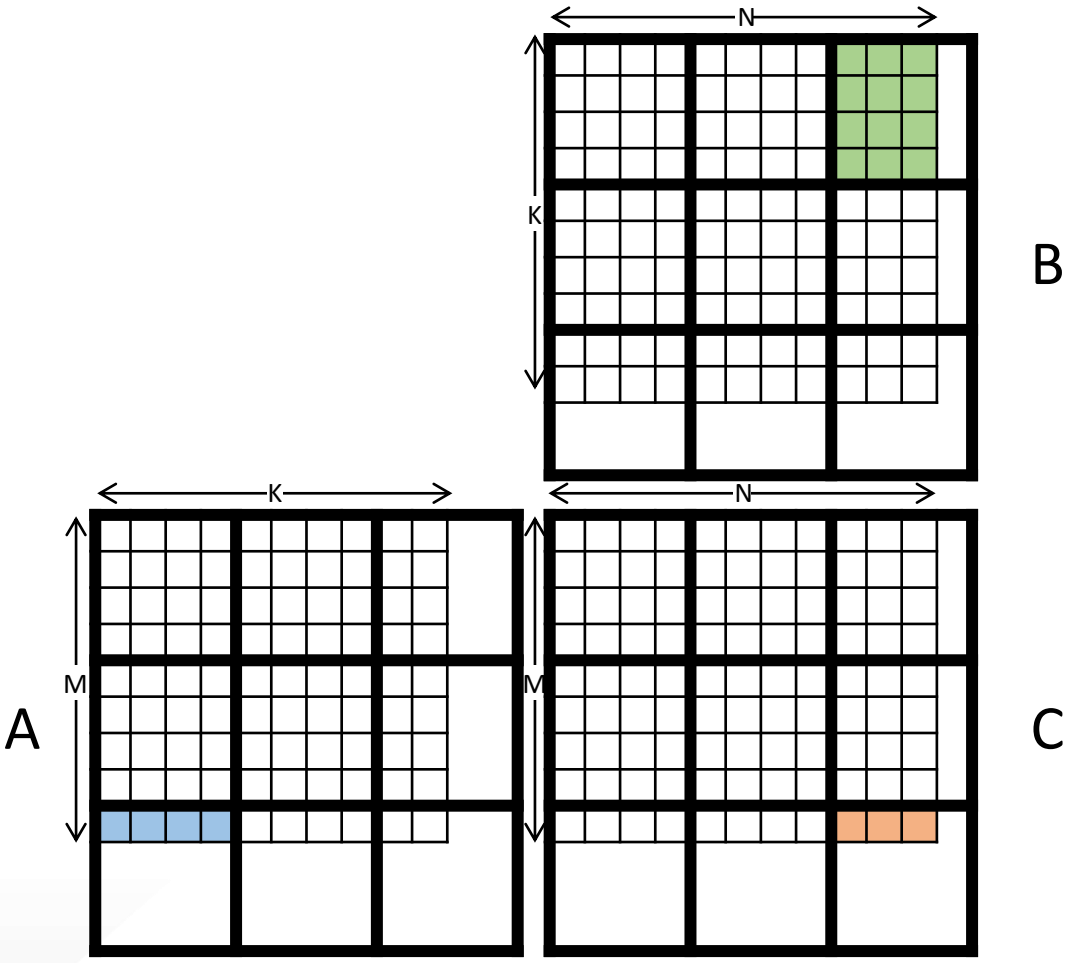
threads active while storing C tile



Different boundary conditions for different accesses (if a thread does not compute an output, this does not mean it is not needed for loading an input)

Copyright © 2022 Elsevier

Different Matrix Dimensions



Copyright © 2022 Elsevier

Tiling on CPU

- Tiling also works for CPU
  - No scratchpad memory, but relies on caches
  - Cache is sufficiently reliable because there are fewer threads running on the core and the cache is larger

```
for(unsigned int rowTile = 0; rowTile < N/TILE_DIM; ++rowTile) {
    for(unsigned int colTile = 0; colTile < N/TILE_DIM; ++colTile) {
        for(unsigned int iTile = 0; iTile < N/TILE_DIM; ++iTile) {
            for (unsigned int row = rowTile*TILE_DIM; row < (rowTile + 1)*TILE_DIM; ++row) {
                for (unsigned int col = colTile*TILE_DIM; col < (colTile + 1)*TILE_DIM; ++col) {
                    float sum = 0.0f;
                    for(unsigned int i = iTile*TILE_DIM; i < (iTile + 1)*TILE_DIM; ++i) {
                        sum += A[row*N + i]*B[i*N + col];
                    }
                    if(iTile == 0) {
                        C[row*N + col] = sum;
                    } else {
                        C[row*N + col] += sum;
                    }
                }
            }
        }
    }
}
```

Copyright © 2022 Elsevier

Memory and Occupancy

GPU Name	K40	M40	P100	V100
Microarchitecture	Kepler	Maxwell	Pascal	Volta
Compute Capability	3.5	5.2	6.0	7.0
Number of SMs	15	24	56	80
Cores per SM	192	128	64	64
Max threads per SM	2048	2048	2048	2048
Max blocks per SM	16	32	32	32
Max threads per block	1024	1024	1024	1024
Registers per SM	64K	64K	64K	64K
Shared memory per SM	48KB	96KB	64KB	96KB

Register usage per thread, and shared memory usage per thread block constrain occupancy

Copyright © 2022 Elsevier

Dynamic Shared Memory

- Shared memory can be dynamically allocated if programmer does not know how much the need beforehand

- Declaration:

```
extern __shared__ A_s[];
```

- Configuration:

```
kernel <<< numBlocks, numThreadsPerBlock, smemPerBlock >>> (...)
```

Copyright © 2022 Elsevier

References

- Wen-mei W. Hwu, David B. Kirk, and Izzat El Hajj. *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 2022.

Copyright © 2022 Elsevier